

HOME BAKERS

Complete Implementation & Repair Roadmap

Prepared from the three uploaded merged codebases dated 11 September 2026

Purpose: This document is the implementation specification for an AI coding agent. It is based on the actual contents of the uploaded Customer React Native app, Admin React Native app, and Node/Express/Prisma/PostgreSQL backend. The agent must implement the missing work in the existing codebase; it must not rebuild the applications from scratch.

Critical rule: A feature is considered complete only when UI → API → backend → database → response → UI works and has been tested. Creating a screen, Prisma model, or API route alone is not completion.

1. SOURCE CODEBASES AUDITED

System	Codebase	Relevant source inventory
Customer	Cakedeliveryapp	71 non-generated/source/config file blocks identified in the merged source export
Admin	cakedeliveryadmin	65 non-generated/source/config file blocks identified in the merged source export
Backend	Backendofbakersapp	40 non-generated/source/config file blocks identified in the merged source export

The merged exports also contain large generated Android/CMake/build artifacts. Those artifacts should not be treated as application source files when deciding feature completeness.

2. EXECUTIVE FINDINGS — WHAT IS STILL MISSING/BROKEN

- Customer app:** the project contains real API service scaffolding, but several screens and flows still contain mock/local/simulated behavior. The existing customer analysis file itself states the app was about 85% functional and that production backend integration was still required.
- Admin app:** dashboard, catalog, onboarding, security, notifications, orders and payment screens exist, but several actions are still alerts/placeholders or use inconsistent API/status behavior.
- Backend:** routes and Prisma models exist, but important relation, authorization, status, payment, onboarding, tracking and endpoint-contract problems remain.
- Cross-app integration:** the biggest remaining task is to make Customer ↔ Backend ↔ Admin use one consistent data model and one real business flow.
- Build safety:** the previous AI editing already introduced syntax mistakes in Dashboardpage.js; therefore every modified file must be syntax/bundle checked immediately after editing.

3. CUSTOMER APP — REQUIRED IMPLEMENTATION

Authentication: Make signup/login persist the returned JWT and user data; every protected request must use the token. Implement or remove unsupported forgot/reset-password API calls because customerApi.js currently references /api/auth/forgot-password and /api/auth/reset-password/:token while the backend route inventory does not expose those endpoints.

API configuration: Use one API base URL. The current config uses 10.0.3.1 in development and production api.homebakers.com. Verify the actual device/emulator environment and remove any remaining hardcoded addresses.

Home/catalog: Replace mock bakery/food/promo/reorder data with backend data where the feature is intended to be live. Keep only genuine UI fallback/empty states, not invented business data.

Category/product: Fix category naming and navigation. Ensure Category, CategoryProducts, CakeDetails and MenuCard agree on props and real product fields.

Cart: Use backend cart for add/update/remove/count. Remove deprecated local-cart functions. Validate quantity and stock on the backend.

Address: Complete add/list/update/delete/default-address behavior. Make the authenticated user own the address records. Fix any old navigation names such as Adressscreen or unrelated Blog navigation.

Checkout: Implement real checkout using the backend. Never trust frontend total. Backend must calculate from current database prices, verify stock, validate address, create order/items/payment state in a transaction and clear cart only after success.

Orders: Replace hardcoded ORDERS data with /api/orders/customer. Add correct order details, statuses, payment state and error/loading/empty states.

Tracking: Replace hardcoded courier/tracking UI with real /api/orders/:orderId/tracking data. Ensure admin updates appear for the customer.

Notifications: Use the real notification API. Fix Notification.js imports/naming and user scoping.

Reviews: Submit and fetch reviews through backend. Prevent arbitrary userId manipulation.

Profile: Connect profile update and password change to authenticated backend identity. Do not trust a userId supplied by the client.

Custom orders: Trace the Customorderpage → Delivery/summary flow and connect it to the actual supported backend model. If the backend has no custom-order persistence, implement the required model/route rather than pretending it was saved.

4. ADMIN APP — REQUIRED IMPLEMENTATION

Dashboard: Fix syntax first, then replace fake metrics. The current Dashboardpage.js contains hardcoded success/future behavior for Download, Coupons, Reports and Restock, and sales-growth calculations that are not real. Implement real backend aggregations or remove misleading actions.

Catalog: Complete create/edit/delete/availability/image upload. Verify every fetch endpoint, auth header, response shape and image URL. Remove dummy data.

Orders: Connect Ordermanagementpage to the real order API. The current cancel request uses /api/orders/:id/cancel, but the backend route inventory does not expose this endpoint. Either implement a proper cancel endpoint or use the canonical status endpoint.

Order status: Standardize the entire system on the database's canonical statuses: Pending, Accepted, Preparing, Ready, Delivered, Cancelled, Rejected. Do not mix lowercase values or out_for_delivery/completed unless a deliberate mapping layer is implemented.

Tracking: Admin tracking must persist through the backend and database. Fix the DeliveryTracking schema/logic so the upsert key is actually unique.

Onboarding: The onboarding wizard must save, fetch and edit the same store/business profile. The current backend creates Onboarding records while /api/store reads StoreProfile, so the agent must reconcile this instead of leaving two disconnected stores.

Security: Delete-account, two-factor and download-data actions must not display success when only a local alert occurred. Implement real supported backend operations or clearly mark external blockers.

Notifications: Remove dummy/fallback notifications and simulated-event buttons for production behavior. Use real notification records.

Profile: Persist profile edits and password changes through authenticated API calls.

Payment: The existing UPI route is a stub. Do not claim payment success without provider verification. Keep payment state accurate and separate from actual gateway confirmation.

Navigation: Audit every navigation target. The merged code shows targets such as Dashboard, Ordermanage, Profile and Securitypage that must match the registered/nested navigator names.

5. BACKEND / DATABASE — REQUIRED IMPLEMENTATION

Prisma schema: The schema contains User, Address, Cart, DeliveryTracking, DeliveryBoy, Discount, Notifications, OrderItem, Order, Payment, Product, Onboarding, StoreProfile and Review.

Review relation bug: Review has userId but the Prisma schema shown does not define a user relation. Reviewcontrol.js uses include: { user: ... }. This must be reconciled before declaring Prisma healthy.

DeliveryTracking relation bug: DeliveryTracking uses upsert where: { orderId }, but orderId is not shown as @unique. Prisma upsert requires a unique selector. Add the correct uniqueness or use another valid unique key, and reconcile the database.

Order ownership: Order currently has no userId relation. getOrderById explicitly leaves the customer authorization check effectively unrestricted. Add a proper customer/user ownership relation or a secure equivalent, migrate it safely, and enforce it in customer order retrieval.

Order status: Database dump constrains order_status to Pending/Accepted/Preparing/Ready/Delivered/Cancelled/Rejected. Backend create/update currently uses lowercase values. This is a direct data-integrity risk and must be fixed everywhere.

Payment state: createOrder creates a Payment with paidAt = new Date() even when paymentStatus is pending. Do not mark an unpaid order as paid or set paidAt unless payment was actually confirmed.

Stock validation: createOrder checks stock only under !prod.publicCatalog, which is logically wrong for a public product. Stock must be validated regardless of catalog visibility, and decrement must be safe against races.

Cart validation: Checkout receives an items array but the server loads the cart separately and does not prove that every submitted item corresponds to the user's current cart. Reconcile checkout input with the authenticated user's cart.

Onboarding/store: saveOnboarding creates an Onboarding row, while getStoreProfile reads StoreProfile. Decide the canonical store profile model and implement create/update/read consistently.

Authentication: authMiddleware currently has a fallback string 'default_secret_change_in_env'. Remove insecure secret fallbacks. Require a real environment secret in production and ensure admin/customer roles are correct.

Profile/password authorization: Profilecontrol.js accepts userId from req.body for password change. Use req.user.userId instead. The same principle applies to profile updates.

Notification/review access: Ensure create/read/mark-read operations use authenticated identity and cannot target another user's records.

Routes: The backend exposes products, cart, orders, tracking, addresses, profile, analytics, notifications, reviews, UPI, store and onboarding. Customer service also calls forgot/reset-password and cancel-order routes that are not present; reconcile these contracts.

6. CURRENT BACKEND ROUTE CONTRACT TO VERIFY

Area	Current route(s) from merged backend	Required action
Auth	/api/auth/signupmain, /api/auth/adminsignup, /api/auth/loginmain, /api/auth/logoutmain	Keep /api/auth/reset-password routes only if required by the app.
Products	/api/products, /search, /category, /:id, POST/PUT/DELETE/PATCH	Verify public/admin authorization and field contract.
Cart	/api/cart/add, GET /api/cart, PUT/DELETE /api/cart/:id	Verify user ownership and stock/quantity validation.
Orders	POST /api/orders, GET /api/orders, /customer, /:id, /:id/status	Add/repair cancel route if cancellation is a required feature.
Tracking	GET/PUT /api/orders/:orderId/tracking	Fix unique/upsert model and permissions.
Address	POST /api/address/save, GET /api/address, PUT/DELETE /api/address/:id	Add default-address behavior if required by UI.
Profile	GET /api/user/profile, PUT /api/user/profile/:id, PUT /api/user/change-password	Use SWD identity; validate IDs.
Analytics	GET /api/dashboard/analytics	Return real metrics including sales-overview data if dashboard requires it.
Notifications	POST/GET /api/notifications, PATCH /:id/read	Enforce user scoping.
Reviews	POST/GET /api/reviews	Fix missing Prisma user relation and authorization.
Payment	POST /api/upi/save	Treat as stub unless real gateway verification exists.

Area	Current route(s) from merged backend	Required action
Store/Onboarding	GET /api/store, POST /api/onboarding/save	Reconcile Onboarding vs StoreProfile.

7. EXACT STEP-BY-STEP IMPLEMENTATION ORDER

- Step 1. Freeze the current working UI and create a backup/git checkpoint before large changes.
- Step 2. Read this PDF and the three merged source exports. Build a feature matrix before coding.
- Step 3. Fix project-level syntax/import/navigation errors first so both React Native apps can bundle.
- Step 4. Backend foundation: validate environment variables, Prisma configuration, database connectivity, migrations and generated client.
- Step 5. Database reconciliation: fix Prisma relations, unique constraints, ownership fields, status constraints and required models. Run format → validate → migration → generate.
- Step 6. Authentication: make customer/admin login return consistent JWT payloads and enforce authenticate/requireAdmin on protected routes.
- Step 7. Customer API layer: verify every service endpoint against backend routes. Remove calls to nonexistent endpoints or implement those endpoints.
- Step 8. Product/catalog: make customer browsing and admin CRUD use the same Product model and response fields.
- Step 9. Cart: make cart fully server-backed, user-scoped and stock-aware.
- Step 10. Address/profile: complete authenticated CRUD and default-address behavior.
- Step 11. Checkout/orders: implement a transactional server-side checkout with authoritative pricing, stock checks, order items, payment state and cart clearing.
- Step 12. Order lifecycle: standardize statuses and implement admin updates plus customer visibility.
- Step 13. Tracking: repair schema/upsert behavior and connect admin updates to customer tracking.
- Step 14. Notifications/reviews: connect both apps to real records and enforce user ownership.
- Step 15. Onboarding/store: choose one canonical persisted store profile flow and connect all onboarding pages to it.
- Step 16. Admin dashboard: implement real analytics, sales overview, best sellers, low-stock metrics and recent reviews. Remove fake numbers.
- Step 17. Admin secondary features: catalog, security, reports/download, coupons, notifications and profile. Every button must either work or be removed/clearly documented as externally blocked.
- Step 18. Payment: preserve honest pending/paid/failed/refunded states. Do not simulate gateway confirmation.
- Step 19. Cross-app testing: customer creates order → admin sees it → admin updates status → customer sees status → tracking updates → review.
- Step 20. Final global audit: search for TODO/FIXME/mock/dummy/fake/coming soon/hardcoded IP/localhost/Alert success/undefined/broken imports/navigation.
- Step 21. Run final builds/tests for backend, customer app and admin app. Fix every error and rerun.
- Step 22. Only then mark the feature matrix green.

8. FEATURE COMPLETION GATES

Feature	Minimum completion gate
Authentication	Signup/login/logout works; token persisted; protected API calls authenticated; admin role enforced.
Products	Customer sees DB products; admin CRUD persists; images resolve; availability/stock are real.
Cart	Add/update/remove/count persist in DB and are isolated per user.
Address	CRUD/default selection persists and cannot access another user's addresses.
Checkout	Server calculates total; validates stock/address; creates order/items/payment state transactionally; clears cart only on success.
Orders	Customer and admin see same DB order; canonical status lifecycle works.

Feature	Minimum completion gate
Tracking	Admin update persists and customer can read it.
Notifications	Real DB records, user-scoped, read state persists.
Reviews	Create/read works; Prisma relation and user identity are valid.
Onboarding	Wizard saves to canonical DB model; data can be retrieved and edited.
Dashboard	All displayed metrics are real; no fabricated growth/sales values.
Security	No fake success, insecure secret fallback, unauthorized data access or password exposure.

9. TEST COMMANDS / VERIFICATION

```
cd Backendofbakersapp npx prisma format npx prisma validate npx prisma migrate status npx prisma generate npm run dev
cd Cakedeliveryapp npx react-native start --reset-cache npx react-native run-android
cd cakedeliveryadmin npx react-native start --reset-cache npx react-native run-android
```

Use the project's actual package scripts when they differ. Do not claim a test passed unless it was actually executed.

10. MASTER INSTRUCTION FOR THE AI CODING AGENT

Paste the following instruction to the coding agent together with this PDF:

You are the lead engineer for the existing Home Bakers project. Read the attached Home Bakers Complete Implementation & Repair Roadmap PDF completely before editing. Also inspect the actual three existing codebases: Cakedeliveryapp, cakedeliveryadmin, and Backendofbakersapp. Your mission is to IMPLEMENT EVERY REQUIRED FEATURE described in the PDF inside the EXISTING codebase. Do not rebuild the applications. Do not create fake screens, fake APIs, fake success messages, mock business data, or placeholder implementations. Work in this exact loop: 1. Inspect the current implementation. 2. Mark the feature/file as NOT IMPLEMENTED, PARTIAL, BROKEN, or COMPLETE. 3. Fix the root cause. 4. Run the relevant syntax/type/bundle/API/database check. 5. Fix any newly exposed issue. 6. Retest. 7. Only then move to the next feature. You must work through Customer app, Admin app, Backend, Prisma/database and cross-app integration. Every frontend API call must be matched against an actual backend route, method, authentication requirement, request body and response shape. Critical known issues to fix include: - Customer mock/local/simulated data and checkout/order/tracking/notification flows. - Missing customer reset-password/cancel endpoints or their incorrect client references. - Customer navigation and screen-name inconsistencies. - Admin Dashboard syntax errors and fake metrics/actions. - Admin order cancellation endpoint mismatch. - Admin dummy notifications/security/report actions. - Backend Review user relation mismatch. - DeliveryTracking upsert uniqueness problem. - Missing secure customer order ownership. - Lowercase backend order statuses versus database canonical statuses. - Incorrect stock condition in checkout. - Payment marked paid/paidAt even when pending. - Onboarding writes one model while store reads another. - Insecure JWT secret fallback. - Profile/password operations trusting client-supplied userId. - Missing/incorrect auth and authorization on protected operations. Do not declare completion after a successful build alone. Perform end-to-end tests: Customer signup/login → browse → product → cart → address → checkout → order → tracking → notification → review. Admin login → dashboard → catalog → product CRUD → order → status update → tracking → notifications → onboarding/security. Then verify the cross-system flow through the same PostgreSQL database. At the end, produce a feature matrix showing every PDF feature, exact files changed, endpoint(s), database model(s), test performed and result. Any item without an actual implementation and test result is NOT complete. If an external credential/provider is genuinely required, document it as an external blocker instead of faking the result. START NOW. Do not only give me a plan. Modify the existing codebase, test it, fix it, and continue until all required features are genuinely complete.

11. FINAL DEFINITION OF DONE

The project is DONE only when: every required feature from the PDF is implemented in the existing applications; every required API is connected; database persistence is correct; authentication/authorization is enforced; customer and admin see the same real state; there are no misleading fake actions/data; Prisma validates and generates successfully; the backend starts; both React Native apps bundle/build; and the critical customer → backend → admin order lifecycle has been tested end-to-end.

12. SOURCE-GROUNDED NOTES

Customer source export: Cakedeliveryapp_merged_2026-09-11_15-44-46.md. Its included analysis explicitly identifies hardcoded IPs, mock data, local checkout, broken navigation/imports and incomplete backend integration.

Admin source export: cakedeliveryadmin_merged_2026-09-11_15-45-26.md. Static inspection shows dashboard/catalog/onboarding/security/notification/order areas with placeholder or simulated actions and multiple navigation/API contracts that require verification.

Backend source export: Backendofbakersapp_merged_2026-09-11_15-45-53.md. The schema, routes and controllers were inspected for the roadmap; important issues include relation mismatches, order-status inconsistency, tracking uniqueness, onboarding/store separation and authorization gaps.